理解 KV 缓存

Understanding KV Cache

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
KV Cache	键值缓存	KV Cache
Inference	推理	
Attention	注意力机制	Attention
Transformer	Transformer 模型	Transformer
Values	值	
Keys	键	
Head Dimension	头维度	
KV Heads	键值头	
Tokens	词元	
MQA	多查询注意力	MQA
GQA	分组查询注意力	GQA
MLA	多头潜在注意力	MLA
Serving	推理服务	
PagedAttention	分页注意力	
Prefix Caching	前缀缓存	
Quantization	量化	
Offloading	卸载	
Prefill	预填充	
Decode	解码	
Generation	生成	
Decoder	解码器	
Token	词元	
Model	模型	
Causal Mask	因果掩码	
Self-attention	自注意力	
Representation	表示	
Query	查询	Q

英文原词	中文译名	缩写／符号
Layer State	层状态	
Matrix	矩阵	
Value Heads	值头	
Batch Size	批大小	
Mental Model	理解框架	
Dense Attention	稠密注意力	
Tensor Payload	张量有效载荷	
MHA	多头注意力	MHA
Tensor	张量	
Models	模型	
Resident	驻留	
Memory Traffic	内存流量	
Kernel	内核	
Incremental Decoding	增量解码	
Multi-head Attention	多头注意力	MHA
Baseline	基线	
Query Head	查询头	
Query Heads	查询头	
Key Heads	键头	
Multi-query Attention	多查询注意力	MQA
Heads	头	
Grouped-query Attention	分组查询注意力	GQA
Multi-head Latent Attention	多头潜在注意力	MLA
Projection Absorption	投影吸收	
Latent Compression	潜在压缩	
Low-rank	低秩	
RoPE	旋转位置嵌入	RoPE
Vector	向量	
Embeddings	嵌入向量	

英文原词	中文译名	缩写／符号
Queries	查询	
Decoupled Positional Component	解耦的位置分量	
Cache Capacity	缓存容量	
Fragmentation	碎片化	
FlashAttention	FlashAttention（方法名保留）	
Block Table	块表	
Block Sharing	块共享	
Packing	打包	
Cache Key	缓存键	
Latency	延迟	
cache_salt	缓存盐值	
Eviction	逐出	
Host Memory	主机内存	
Data Movement	数据移动	
Precision	精度	
Calibration	校准	
Throughput	吞吐量	
Benchmark	基准测试	
Accuracy	准确性	
Quantization Error	量化误差	
Activation	激活	
Dtype	数据类型	
Efficient Attention	高效注意力	
Memory Bandwidth	内存带宽	
Recomputation	重计算	
Numerical Precision	数值精度	
Causal Attention	因果注意力	
Training	训练	
Mixture-of-Experts	混合专家	MoE

英文原词	中文译名	缩写／符号
Large Language Model	大语言模型	LLM
Quantized	量化后的	

目录导航

封面与说明 · 第 1 页
专业术语翻译对照表 · 第 2 页
理解 KV 缓存 · 第 7 页
本手册将澄清哪些问题 · 第 8 页
为什么解码是顺序的 · 第 9 页
朴素的解码器（Decoder）会重复什么 · 第 10 页
到底缓存了什么？ · 第 11 页
缓存不是单个矩阵（Matrix） · 第 12 页
两个阶段的缓存创建方式不同 · 第 13 页
四个词元（Token）的示例 · 第 14 页
键值缓存（KV Cache）有多大？ · 第 15 页
在同一模型（Model）中比较 MHA、GQA 与 MQA · 第 16 页
为什么长期背景变化推理服务（Serving）于经济学 · 第 17 页
它不保存什么 · 第 18 页
KV 状态的昂贵基线（Baseline） · 第 19 页
跨查询头（Query Heads）共享 K/V · 第 20 页
MHA 和 MQA 之间的中间点 · 第 21 页
为什么 KV 头数如此重要 · 第 22 页
压缩表示（Representation）而不是仅共享头 · 第 23 页
RoPE 和缓存密钥会发生什么情况？ · 第 24 页
为什么缓存管理变得困难 · 第 25 页
为什么连续分配会浪费容量 · 第 26 页
逻辑顺序，不连续的物理块 · 第 27 页
分页有其自身的权衡 · 第 28 页
跨请求重用预填充（Prefill）状态 · 第 29 页
它减少了预填充（Prefill）工作，而不是未来的解码（Decode）工作 · 第 30 页
重用可以通过计时揭示信息 · 第 31 页
当 GPU 缓存已满时会发生什么？ · 第 32 页
使用较慢的内存作为较大的缓存层 · 第 33 页
用更少的位数存储 K/V · 第 34 页
压缩改变数值状态 · 第 35 页
创造错误思维模式的捷径 · 第 36 页
关于任何 KV 缓存设计要问的五个问题 · 第 37 页
用一条因果链理解 键值缓存（KV Cache） · 第 38 页
主要论文和当前实施来源 · 第 39 页

理解 KV 缓存

对应原稿第 1 页

了解 键值缓存（KV Cache）

从自回归解码到现代 LLM 推理（Inference）

面向人工智能工程师、开发人员和认真学习者的简短技术手册

@techNmak

手册02 - 理解AI系列

本手册将澄清哪些问题

对应原稿第 2 页

阅读导引

键值缓存（KV Cache）很容易被描述得不准确。人们常把它称为“大语言模型的记忆”“加速技巧”，或简单称为“缓存的注意力机制（Attention）”。这些简略说法掩盖了真正重要的机制。

本手册从基本原理出发构建了这一想法。到最后，您应该能够解释：

• 为什么过去的键和值可以在普通因果 Transformer 中重用；

• 确切存储哪些张量以及它们的形状如何增长；

• 为什么缓存内存会随着缓存的词元（Tokens）、层、KV 头、头维度（Head Dimension）和每个元素的字节数线性扩展；

• 为什么MQA、GQA和MLA以不同的方式攻击KV成本？

• 为什么缓存管理成为并发推理服务（Serving）下的系统问题？

• PagedAttention、前缀缓存（Prefix Caching）、卸载（Offloading）和KV 缓存量化（Quantization）实际发生了什么变化。

先备知识

你应该已经知道基本的注意力机制（Attention）方程以及 Q、K 和 V 的作用。之前的手册《理解注意力机制》是自然的前提。

目标不是再记住一种优化，而是理解其数学有效性、节省的资源以及仍然存在的成本。

研究依据：[1]、[2]、[5]、[6]

为什么解码是顺序的

对应原稿第 3 页

1 - 自回归生成（Generation）

仅解码器（Decoder）语言模型（Model）根据已有词元（Tokens），为下一个词元给出条件概率分布：

$$p(x_{t+1} \mid x_{1:t}).$$

选择词元（Token）后，它会被附加到序列中，并成为下一步上下文的一部分。因此，生成（Generation）形成了一个依赖链：

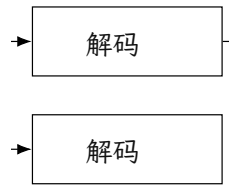
$$x_1, \ldots, x_t \to x_{t+1} \to x_{t+2} \to \cdots$$

两个执行阶段

对于许多词元（Tokens）的提示，模型（Model）可以在因果掩码（Causal Mask）下一起处理这些提示位置。这通常称为预填充（Prefill）。一旦生成（Generation）开始，模型通常会在每个解码（Decode）步骤的每个序列中推进一个新生成的词元。这是解码阶段。



预填充（Prefill）



在这种顺序解码（Decode）结构中，KV 缓存变得至关重要。

研究依据：[1]、[6]

朴素的解码器（Decoder）会重复什么

对应原稿第 4 页

2 - 冗余计算

考虑一个自注意力（Self-attention）层。对于每个词元（Token）表示（Representation），该层形成学习投影：

$$q_i = h_i W_Q, \qquad k_i = h_i W_K, \qquad v_i = h_i W_V.$$

在解码（Decode）步骤 t,中，新查询（Query）必须关注位置$1, \ldots, t.$的键和值

简单的实现可以在每一步中从头开始重建旧的 k_i和 v_i张量。这将重复执行输入未更改的投影。

关键观察

对于普通的因果 Transformer，一旦在固定层处理了位置 i，添加未来的词元（Tokens）不会追溯改变该位置的层状态（Layer State）。因果掩码可防止来自未来位置的信息向后流动到较早词元的表示（Representation）中。

因此，可以保留从较早位置导出的键和值，并在以后的解码（Decode）步骤中重用。

KV 缓存是有效的，因为在模型（Model）的因果计算下，过去的层状态对于未来的词元（Tokens）是不变的。这不是一个启发式的猜测。

研究依据：[1]、[6]

到底缓存了什么？

对应原稿第 5 页

3 - 机制

假设模型（Model）已经处理了位置$1, \ldots, t-1$.在一个注意力机制（Attention）层，它已经存储

$$K_{1:t-1} = [k_1; \ldots; k_{t-1}], \qquad V_{1:t-1} = [v_1; \ldots; v_{t-1}].$$

对于新位置 t,，该层仅计算新投影

$$q_t, \qquad k_t, \qquad v_t,$$

然后附加新的键和值：

$$K_{1:t} = [K_{1:t-1}; k_t], \qquad V_{1:t} = [V_{1:t-1}; v_t].$$

当前查询（Query）对累积的历史执行注意力计算：

$$\text{softmax}\left(\frac{q_t K_{1:t}^T}{\sqrt{d_h}}\right) V_{1:t}.$$

这一机制没有缓存什么

缓存不存储“思想”或符号化事实。它存储每个注意力机制（Attention）层的数值键／值状态和缓存的词元（Token）位置，无论推理（Inference）引擎使用什么物理布局。

研究依据：[1]、[6]

缓存不是单个矩阵（Matrix）

对应原稿第 6 页

4 - 每个注意力机制（Attention）层一个缓存

每个自注意力（Self-attention）层都有自己的学习投影，因此也有自己的 K/V 状态。具有 L注意力机制（Attention）层的模型（Model）维护如下逻辑历史：

$$\boxed{\text{Layer } 1\text{: } K^{(1)}, V^{(1)}}$$

层1: $K^{(1)}, V^{(1)}$

$$\boxed{\text{Layer } 2\text{: } K^{(2)}, V^{(2)}}$$

层2: $K^{(2)}, V^{(2)}$

$$\vdots$$

$$\boxed{\text{Layer } L\text{: } K^{(L)}, V^{(L)}}$$

层 L: $K^{(L)}, V^{(L)}$

对于常见的逻辑布局，一层的缓存可以被认为是

$$K, V \in \mathbb{R}^{B \times H_{KV} \times T \times D_h},$$

其中 B是批大小（Batch Size）， H_{KV}是键/值头（Value Heads）的数量， T是缓存序列长度，D_h是头维度（Head Dimension）。

实际的内核可能会以不同的方式存储或打包尺寸。逻辑形状是重要的理解框架（Mental Model）；特定的物理布局是实现细节。

研究依据：[2]、[6]

两个阶段的缓存创建方式不同

对应原稿第 7 页

5 - 预填充（Prefill）和解码（Decode）

预填充（**Prefill**）

给定输入提示，模型（Model）会在每一层并行计算许多提示位置的 K/V 状态，并受到因果掩码的影响。这些状态填充初始缓存。

解码（**Decode**）

对于每个新生成的词元（Token），每一层都会计算一个新的 q_t, k_t, v_t，追加 k_t, v_t，并根据缓存的历史记录对新查询（Query）进行关注。

	Prefill	Decode
New positions processed	Many prompt tokens	Usually one new token per sequence/step
Cache action	Build initial K/V history	Append new K/V state
Attention query length	Many	Usually 1
Reuse benefit	Establishes state	Avoids recomputing past K/V projections

新处理的位置／缓存操作／注意力机制（Attention）长度／复用收益

位置

查询（Query）

预填充（Prefill）

许多提示词元（Tokens）

建立初始 K/V 历史 许多

建立初始状态

解码（Decode）

通常每个序列/步骤一个新词元（Token） 附加新的 K/V 状态 通常 1

避免重新计算过去的 K/V 预测

键值缓存（KV Cache）并没有消除对历史内容的注意力机制（Attention）计算。对于稠密注意力（Dense Attention），新查询（Query）仍须与过去的 K/V 状态交互。

研究依据：[6]、[7]

四个词元（Token）的示例

对应原稿第 8 页

6 - 逐步解码（Decode）

想象一下缓存当前保存位置 1-3。下一步：

1. 模型（Model）在这一层接收词元（Token） 4 的表示（Representation）。

2. 计算 q_4, k_4,和 v_4.

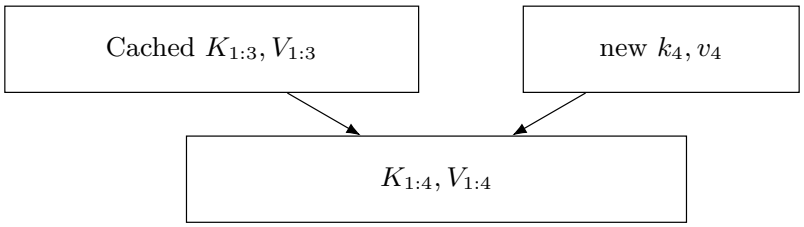
3. 将 k_4和 v_4附加到层缓存中。

4. q_4与 $K_{1:4}$.比较

5. 得到的注意力机制（Attention）权重对 $V_{1:4}$.进行加权组合。

6. 该层生成模型（Model）其余部分使用的表示（Representation）。

旧的 k_1, k_2, k_3和 v_1, v_2, v_3只需读取，无须重新生成。



已缓存 $K_{1:3}, V_{1:3}$

新 k_4, v_4

研究依据：[6]

键值缓存（KV Cache）有多大?

对应原稿第 9 页

7 - 记忆公式

对于具有统一 KV 头计数和头维度（Head Dimension）的传统稠密注意力（Dense Attention）堆栈，理想化的每序列缓存大小为

$$M_{KV} = 2\,L\,T\,H_{KV}\,D_h\,b$$

其中：

• 系数 2 表示同时存储 K 和 V；

- L为注意力机制（Attention）层数；
- T是缓存的词元（Tokens）；
- H_{KV}是键/值头（Value Heads）的数量；
- D_h为头维度（Head Dimension）；
- b是每个存储元素的字节数。

对于 B等长序列，乘以 B.更一般地，对于可变缓存长度 T_i,，理想化的有效载荷为

$$M_{KV} = 2\,L\,H_{KV}\,D_h\,b\sum_{i=1}^{B} T_i.$$

真正的推理服务（Serving）引擎以块的形式进行分配，并且可能添加填充、缩放、元数据、分片或特定于模型（Model）的状态，因此这些公式描述的是张量有效载荷（Tensor Payload），而不是运行时消耗的每个字节。

该公式很有用，因为每个架构选择都有一个可见的杠杆：上下文长度变化 T;

MQA/GQA 更改 H_{KV};缓存量化（Quantization）更改存储项 b（加上量化元数据）。

研究依据：[2]、[3]、[4]、[5]

在同一模型（Model）中比较 MHA、GQA 与 MQA

对应原稿第 10 页

8 - 工作记忆示例

采取一个说明性的解码器（Decoder）

$$L = 32, \quad D_h = 128, \quad b = 2 \text{ bytes}, \quad H_Q = 32.$$

我们唯一改变的值是 H_{KV}.

架构	H_{KV}	KiB / 词元	GiB at 8,192 tokens
MHA	32	512	4.000
GQA（示例）	8	128	1.000
MQA	1	16	0.125

GiB 为 8,192 个词元（Tokens）

对于 GQA 行：

$$2 \cdot 32 \cdot 8192 \cdot 8 \cdot 128 \cdot 2 = 1{,}073{,}741{,}824 \text{ bytes} = 1 \text{ GiB}.$$

这些是每个序列张量（Tensor）有效载荷的示例，不是所有 32 层模型（Models）的通用大小。不同的模型使用不同的层数、KV 头、维度、数据类型、局部注意力机制（Attention）模式和分片。

研究依据：[2]、[3]

为什么长期背景变化推理服务（Serving）于经济学

对应原稿第 11 页

9 - 上下文长度

保持其他一切固定不变，

$$M_{KV} \propto T.$$

这意味着缓存词元（Tokens）的数量加倍，理想化的 KV 张量有效载荷（Tensor Payload）也加倍。

使用与上一页相同的说明模型（Model）：

架构	8K	32K	128K
MHA ($H_{KV} = 32$)	4 GiB	16GiB	64 GiB
GQA ($H_{KV} = 8$)	1 GiB	4GiB	16 GiB
MQA ($H_{KV} = 1$)	0.125GiB	0.5GiB	2GiB

MHA($H_{KV} = 32$)

GQA($H_{KV} = 8$)

MQA($H_{KV} = 1$)

4 GiB 1 GiB

64 GiB 16 GiB

模型（Model）的参数内存不会仅仅因为一个请求有更多上下文而增长，而是它的每个请求的 KV 状态都会增长。这就是为什么即使模型权重不变，长上下文能力也会产生推理服务（Serving）能力问题。

上下文窗口规范告诉您模型（Model）可以接受什么。它不会告诉您特定服务器可以同时容纳多少个长上下文请求。

研究依据：[5]、[7]

它不保存什么

对应原稿第 12 页

10 - KV 缓存节省了什么

KV 缓存避免重新计算历史键/值投影以及生成它们所需的早期层状态。这大大减少了自回归解码过程中的冗余工作。

但对于普通的稠密注意力（Dense Attention）来说，新的查询（Query）仍然需要访问过去的 K/V 状态。随着 T的增长，新位置的注意力机制（Attention）操作仍然与更长的历史交互。

三种不同的资源

不要将这些内容合并为一项声明：

• 计算：预测和注意力机制（Attention）所需的算术。

• 容量：保持K/V 状态驻留（Resident）所需的字节数。

• 内存流量（Memory Traffic）/带宽：在解码过程中为提供注意力机制（Attention）内核（Kernel）而移动的字节。

MQA 的明显动机是在增量解码（Incremental Decoding）期间重复加载大 K/V 张量的内存带宽成本，而不仅仅是“使用更少的参数”。

研究依据：[2]

KV 状态的昂贵基线（Baseline）

对应原稿第 13 页

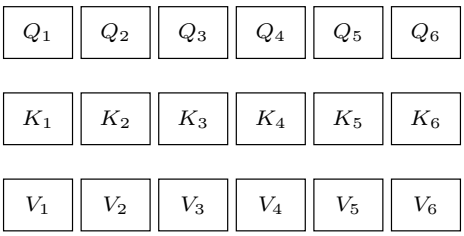
11 - 多头注意力（Multi-head Attention）

在标准多头注意力（Multi-head Attention）中，每个查询头（Query Head）都有相应的键头和值头。从概念上讲：

$$H_{KV} = H_Q.$$

如果有 32 个查询头（Query Heads），则有 32 个键头（Key Heads）和 32 个值头（Value Heads）。

在解码（Decode）期间，每个新的查询头（Query Head）都会关注与其头关联的历史 K/V 状态。这为架构提供了丰富的每头 K/V 投影，但也使得 键值缓存（KV Cache）与 KV 头数量成正比增长。



此页面是 MQA 和 GQA 有意义的基线（Baseline）。

研究依据：[1]、[2]

跨查询头（Query Heads）共享 K/V

对应原稿第 14 页

12 - 多查询注意力（Multi-query Attention）

多查询注意力（Multi-query Attention）（MQA）保留多个查询头（Query Heads），但在它们之间共享一个键头和一个值头：

$$H_Q > 1, \qquad H_{KV} = 1.$$

Noam Shazeer 的 2019 年论文特别围绕增量解码（Incremental Decoding）推动了这一变化：加载 K/V 张量可能成为内存带宽瓶颈，因此缩小这些张量可以提高解码效率。

有什么变化

查询头（Query Heads）保持不同。减少来自 K/V 共享，而不是删除查询头。

缓存公式说明了什么

如果将具有 32 个查询（Query）/KV 头的 MHA 模型（Model）移至 1 个 KV 头，同时保持相同的 $L, D_h, T, b,$，则 KV 张量有效载荷（Tensor Payload）将减少 32 倍。

该算术是架构的结果。特定的 MQA 模型（Model）是否保持相对于特定 MHA 基线（Baseline）的质量是一个经验问题，而不是数学保证。

研究依据：[2]

MHA 和 MQA 之间的中间点

对应原稿第 15 页

13 - 分组查询注意力（Grouped-query Attention）

分组查询注意力（Grouped-query Attention）（GQA）使用的 KV 头比查询头（Query Heads）少，但多于一个：

$$1 < H_{KV} < H_Q.$$

查询头（Query Heads）被分成组，并且组中的查询头共享相同的K/V头。

	MHA	GQA	MQA
Query heads	many	many	many
KV heads	same as Q	intermediate	1
K/V sharing	none across Q heads	within groups	across all Q heads

查询头（Query Heads） KV 头 K/V 共享

Q头没有

与Q相同

许多

组内中间

许多

跨越所有Q头

许多

安斯利等人。介绍了 GQA 作为 MQA 的泛化，并报告说他们的上训练 GQA 模型（Models）在实验中达到了接近 MHA 的质量，速度与 MQA 相当。

“GQA 与 MHA 一样好”过于宽泛。可辩护的声明是关于所报告的模型（Models）和

GQA 论文中的实验设置。

研究依据：[3]

为什么 KV 头数如此重要

对应原稿第 16 页

14 - 一种公式，三种架构

对于固定 $L, T, D_h, b,$

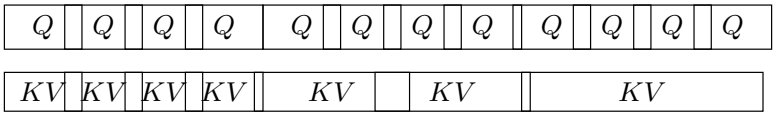
$$M_{KV} \propto H_{KV}.$$

因此，头共享的选择直接在内存消耗中可见。

GQA

MQA

MHA



这就是为什么 键值缓存（KV Cache）不能独立于注意力机制（Attention）架构来讨论。具有相同参数规模和上下文长度的模型（Model）可能具有非常不同的缓存要求，具体取决于其 K/V 表示（Representation）。

研究依据：[2]、[3]

压缩表示（Representation）而不是仅共享头

对应原稿第 17 页

15 - 多头潜在注意力（Multi-head Latent Attention）

DeepSeek-V2 引入了多头潜在注意力（Multi-head Latent Attention）（MLA）作为降低 KV 缓存成本的不同途径。其中心思想是通过低维潜在状态来表示非位置K/V信息。推理（Inference）时，投影吸收（Projection Absorption）让模型（Model）避免以传统方式具体化和缓存完整的每头 K/V 张量。

这在概念上与 MQA/GQA 不同：

• MQA：1个共享K/V头；

• GQA：较少数量的共享K/V头；

• MLA：K/V 信息的低阶潜在压缩（Latent Compression）。

DeepSeek-V2 报告称，相对于其声明的比较架构，KV 缓存减少了 93.3%。该数字是特定于论文的结果，而不是通用的“MLA 压缩比”。

绳索**PE**并发症

DeepSeek-V2 还使用解耦旋转位置组件，因为普通 RoPE 与 MLA 使用的投影吸收策略相互作用。这就是为什么简单的“MLA 缓存一个微小向量（Vector）”摘要是不完整的。

研究依据：[4]

RoPE 和缓存密钥会发生什么情况?

对应原稿第 18 页

16 - 位置信息

通过标准旋转位置嵌入向量（Embeddings），位置相关的旋转应用于查询（Queries）

和键之前，它们的点积被注意力机制（Attention）使用。在普通缓存解码期间，存储以供将来重用的历史密钥必须与后续查询（Queries）所期望的位置处理保持一致。

一个有用的实现级理解框架（Mental Model）是：

• 计算新位置的新密钥；

• 根据注意力机制（Attention）实现的要求应用模型（Model）的位置变换；

• 以未来注意调用所期望的形式存储/重用密钥表示（Representation）。

不同的架构可以准确地改变存储的内容。 MLA 是最明显的例子：与传统的 MHA/GQA 模型（Model）相比，其解耦的位置组件改变了缓存结构。

不要将“键值缓存（KV Cache）始终存储 RoPE 后 K”硬编码为通用定义。缓存表示（Representation）依赖于体系结构和实现。

研究依据：[4]、[6]

为什么缓存管理变得困难

对应原稿第 19 页

17 - 从一个请求到服务器

单个请求有一个简单的故事：它的缓存随着词元（Tokens）的处理而增长。推理服务（Serving）系统有许多具有不同提示长度、生成（Generation）长度、到达时间和完成时间的请求。

在任何时候，服务器可能需要管理：

• 刚刚开始的一个序列；

• 另一个已经持有数以万计的缓存词元（Tokens）；

• 完成并释放内存的多个请求；

• 共享前缀的请求；

• 需要不同注意窗口模式的请求。

这将 键值缓存（KV Cache）从张量（Tensor）变成了动态内存管理问题。

PagedAttention 论文将碎片和冗余重复确定为当代推理服务（Serving）系统中 KV 缓存容量（Cache Capacity）浪费的主要来源，从而激发了受虚拟内存启发的基于块的设计。

研究依据：[5]

为什么连续分配会浪费容量

对应原稿第 20 页

18 - 碎片化（Fragmentation）

如果每个请求都需要一个大的连续 KV 分配，其大小适合其最大可能的序列长度，则可能会出现两种浪费：

保留但未使用的空间

请求可能会保留它从未到达的空间。

外部碎片

空闲内存可能以单独太小的片段存在，无法满足大的连续请求，即使它们的总大小足够。

PagedAttention 设计避免了要求每个逻辑序列的 KV 状态位于一个连续的物理区域中。相反，它将缓存状态划分为固定大小的块，并将逻辑块映射到物理块。

分页是关于内存管理的。它不应该与 FlashAttention 混淆，FlashAttention 主要是一种用于计算注意力机制（Attention）的 IO 感知算法/内核（Kernel）策略。

研究依据：[5]

逻辑顺序，不连续的物理块

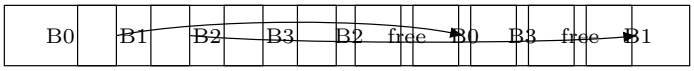
对应原稿第 21 页

19 - 页面注意

PagedAttention 将受操作系统启发的思想应用于 KV 状态。序列被划分为逻辑词元（Token）块。这些逻辑块可以驻留在任意可用的物理 KV 缓存块中。

逻辑顺序

物理缓存



免费

免费

块表（Block Table）告诉注意力机制（Attention）实现每个逻辑块所在的位置。可以随着请求的增长来分配块，并在不再需要时释放块。

最初的 vLLM 作品还使用块共享（Block Sharing）来减少相关序列的重复。

研究依据：[5]

分页有其自身的权衡

对应原稿第 22 页

20 - 块大小

KV 块代表固定范围的词元（Token）头寸。较小的块可以减少序列尾部的内部浪费，并使重用更加细化。较大的块可以减少管理开销，并且可能更好地适合某些内核（Kernel）/内存传输选择。

不存在独立于引擎、硬件、工作负载、模型（Model）和内核（Kernel）设计的通用最佳块大小。

现代推理服务（Serving）系统公开了面向块的缓存管理器，但它们的确切默认值和物理打包（Packing）应被视为实现细节，而不是 KV 缓存定义的一部分。

持久的想法

$$\neq$$

逻辑词元（Token）历史

一个连续的物理分配

这种分离允许运行时以块粒度分配、共享、逐出和移动缓存状态。

研究依据：[5]、[7]

跨请求重用预填充（Prefill）状态

对应原稿第 23 页

21 - 前缀缓存（Prefix Caching）

普通的 KV 缓存会重用正在进行的请求中的早期状态。前缀缓存（Prefix Caching）更进一步：如果后面的请求具有与先前计算的状态匹配的可重用前缀，则服务器可以重用该前缀的缓存 KV 块，而不是重新计算其预填充（Prefill）。

当前的 vLLM 使用基于前缀和块内容的块哈希实现自动前缀缓存（Prefix Caching），并在必要时添加额外输入以确保正确性（例如，模型（Model）/运行时特定上下文）。

受益的两种工作负载

• 对同一个长文档或系统提示重复提问；

• 多轮对话，其中大量历史记录作为前缀重复发送。

前缀缓存（Prefix Caching）是匹配计算前缀的精确状态重用。它不是语义相似性搜索，也不是 RAG。

研究依据：[7]

它减少了预填充（Prefill）工作，而不是未来的解码（Decode）工作

对应原稿第 24 页

22 - 前缀缓存（Prefix Caching）限制

前缀缓存（Prefix Caching）可以跳过先前处理的共享前缀的计算。一旦生成（Generation）超出了该前缀，新的词元（Tokens）仍然必须正常解码。

当前的 vLLM 文档明确指出了这一点：自动前缀缓存（Prefix Caching）减少了查询处理/预填充（Prefill）时间，但不会减少在解码（Decode）阶段生成新词元（Tokens）所需的时间。

这意味着即使其提示前缀是可重用的，由生成的很长答案主导的工作负载也可能看不到什么好处。

正确性需要的不仅仅是匹配文本

安全的缓存密钥可能需要区分的不仅仅是可见的词元（Token）文本：模型（Model）标识和配置、适配器、多模式输入或其他执行状态可能会影响缓存的块是否可重用。确切的要求是特定于运行时的。

研究依据：[7]

重用可以通过计时揭示信息

对应原稿第 25 页

23 - 前缀缓存（Prefix Caching）安全

在多租户服务中，缓存命中可能会改变延迟（Latency）。如果允许不相关的用户共享前缀缓存状态，攻击者可能能够通过观察时间差异来推断某些内容是否已经被缓存。

当前vLLM提供可选的每个请求cache_salt。 salt 被合并到第一个块的缓存标识中，因此只有共享 salt 的请求才能重用这些块，从而为缓存共享创建信任边界。

相同的工程原理比任何一种框架都更广泛：

缓存复用域应与信任域保持一致.

哈希选择也很重要：当前的 vLLM 文档警告说，非加密哈希会增加冲突风险，并可能在共享环境中产生正确性或隐私问题。

研究依据：[7]

当 GPU 缓存已满时会发生什么？

对应原稿第 26 页

24 - 驱逐

生产缓存的容量是有限的。当需要新的 K/V 块时，旧的可重用块可能必须离开 GPU 内存。

不存在单一的普遍驱逐政策。示例包括：

• LRU 式驱逐可重用前缀块；

• 优先级感知的LRU 变体；

• 保护属于主动解码请求的块的策略；

• 可扩展卸载（Offloading）系统中应用程序定义的策略。

当前的 vLLM 前缀缓存设计记录了基于 LRU 的释放/驱逐路径。当前的 TensorRT-LLM 文档描述了可重用块的优先 LRU。

驱逐可重用前缀块不会改变模型（Model）的正确性。它改变了未来的请求是否可以避免重新计算该状态。逐出活动序列所需的缓存是另一回事，必须由运行时协调。

研究依据：[7]、[8]

使用较慢的内存作为较大的缓存层

对应原稿第 27 页

25 - 卸载（Offloading）

当 GPU 内存不足时，可重用的 KV 块可以移动到更大但速度较慢的存储

并在需要时晋升回来。

一个常见的层次结构是

GPU HBM ↔CPU 内存 ↔辅助存储/网络层.

当前的 vLLM 支持 CPU KV 卸载（Offloading）和多层卸载设计，其中辅助层通过 CPU 内存暂存。当前的 TensorRT-LLM 可以将可重用块卸载到主机内存（Host Memory），并公开用于外部 KV 缓存存储或传输的连接器。

权衡

卸载（Offloading）扩展了有效的缓存容量（Cache Capacity），但引入了数据移动（Data Movement）。它是否有帮助取决于重用概率、传输带宽/延迟（Latency）、与计算的重叠以及重新计算状态的成本。

“卸载到 CPU 以节省 GPU 内存”不完整。真正的决定是：传输缓存状态或

重新计算一下？

研究依据：[8]、[9]

用更少的位数存储 K/V

对应原稿第 28 页

26 - KV 缓存量化（Quantization）

减少缓存容量（Cache Capacity）和带宽压力的另一种方法是以较低精度（Precision）存储 K/V 激活。

从概念上讲，如果张量有效载荷（Tensor Payload）从 16 位存储转移到理想的 8 位存储，则 元素存储 组件将减半。实际的实现还需要缩放、对齐、元数据和特定于后端的布局，因此实际的内存节省不需要完全等于理想的位宽比。

当前的 vLLM 支持多种 KV 缓存存储数据类型，包括支持的硬件/后端上的 FP8 变体。它还记录了量化（Quantization）缩放因子的校准（Calibration）路径。

KVQuant 等研究系统甚至可以研究较低位的 KV 表示，但它们报告的准确性（Accuracy）/吞吐量（Throughput）结果是特定于方法、模型（Model）、硬件和基准的。

研究依据：[10]、[11]

压缩改变数值状态

对应原稿第 29 页

27 - 量化（Quantization）不是免费的

与 PagedAttention 的内存放置或精确前缀重用不同，KV 缓存量化（Quantization）改变了缓存张量的数值表示（Representation）。

这将量化误差（Quantization Error）引入到未来的注意力机制（Attention）计算中。工程问题变成：

vs

节省内存/带宽

质量和内核（Kernel）成本.

正确的精度（Precision）取决于：

• 模型（Model）架构和层敏感性；

• 键/值激活（Activation）分布；

• 上下文长度；

• 量化（Quantization）粒度和尺度；

• 硬件支持和可用内核。

当前的 vLLM 甚至允许选定的层或注意力机制（Attention）类型保持未量化，而其他层或注意力机制类型则使用较低精度（Precision）的 KV dtype，这反映了灵敏度不需要统一的事实。

研究依据：[10]、[11]

创造错误思维模式的捷径

对应原稿第 30 页

28 - 常见的误解

“键值缓存（**KV Cache**）是模型（**Model**）的内存。”

不是。它是从注意层的词元（Token）序列导出的临时推理（Inference）状态。

“**KV** 缓存使解码时间恒定。”

不会。它避免了重新计算历史 K/V 状态；对新查询（Query）的高度关注仍然与不断扩展的历史相互作用。

“**MQA/GQA** 减少查询头（**Query Heads**）。”

不。它们的定义变化是相对于查询头（Query Heads）的 K/V 头的数量/共享。

“**PagedAttention** 是 **FlashAttention**。”

不会。PagedAttention 解决的是 KV-cache 内存管理问题； FlashAttention 解决高效的注意力计算/内存流量（Memory Traffic）问题。

“前缀缓存（**Prefix Caching**）可以加快每个生成的词元（**Token**）的速度。”

不会。它可以跳过可重用的前缀预填充（Prefill）；它不会删除新词元（Tokens）的正常解码（Decode）工作。

“**KV**量化（**Quantization**）就是无损压缩。”

一般不会。较低精度（Precision）的表示（Representation）引入了数值近似。

研究依据：[2]、[5]、[7]、[10]

关于任何 KV 缓存设计要问的五个问题

对应原稿第 31 页

29 - 生产理解框架（Mental Model）

1. 存储什么？哪些 K/V 或潜在状态、在哪些层、在什么数据类型（Dtype）/布局中？

2. 每个词元（Token）创建多少状态？ Track L, H_{KV}, D_h,字节、注意窗口规则和分片。

3. 状态住在哪里？连续的 GPU 张量、分页块、CPU 卸载、远程缓存还是多层？

4、什么时候可以重复使用？在请求内、跨匹配前缀、跨实例还是根本不？

5、瓶颈是什么？ GPU 容量、内存带宽（Memory Bandwidth）、传输延迟（Latency）、重新计算、碎片或调度程序压力？

这些问题将数学 KV 缓存机制与围绕其构建的推理服务（Serving）系统分开。

研究依据：[5]、[7]、[8]、[9]

用一条因果链理解 键值缓存（KV Cache）

对应原稿第 32 页

30 - 完整的理解框架（Mental Model）

因果 Transformer 从先前的词元（Tokens）中重复生成新的词元。当附加未来的词元时，过去的注意力机制（Attention）层状态不会改变，因此模型（Model）可以保留它们的键和值，而不是重新计算它们。

对于传统的缓存，张量有效载荷（Tensor Payload）的增长大致为

$$2LTH_{KV}D_hb.$$

这使得上下文长度和并发请求变得昂贵。 MQA/GQA减少KV头数量； MLA 压缩缓存的表示（Representation）；较低精度（Precision）的存储会减少每个元素的字节数。

在推理服务（Serving）规模上，动态序列长度会产生分配和重用问题。基于块的缓存管理将逻辑词元（Token）历史与物理内存放置分开。前缀缓存（Prefix Caching）在兼容请求中重复使用已计算的预填充（Prefill）状态。驱逐和卸载（Offloading）决定哪些可重用块保留在 GPU 附近。量化（Quantization）以数值精度（Numerical Precision）换取容量/带宽节省。

键值缓存（KV Cache）同时是因果注意力（Causal Attention）带来的结果、模型（Model）架构的设计约束，

以及运行时的内存管理问题。理解这三个层面，才能真正掌握这一概念。

研究依据：[1]-[11]

主要论文和当前实施来源

对应原稿第 33 页

参考文献

[1] A. Vaswani et al. 您所需要的就是注意力机制（Attention）。 NeurIPS 2017.arXiv:1706.03762.

原题名与书目信息：[1] A. Vaswani et al. Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.

[2] N. Shazeer. 快速 Transformer 解码：一个写头即可满足您的需求。 arXiv:1911.02150，2019。

原题名与书目信息：[2] N. Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150, 2019.

[3] J. Ainslie et al. GQA：从多头检查点训练（Training）广义多查询（Query）变压器模型（Models）。 EMNLP 2023.arXiv:2305.13245.

原题名与书目信息：[3] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP 2023. arXiv:2305.13245.

[4] DeepSeek-AI et al. DeepSeek-V2：强大、经济且高效的专家混合语言模型（Model）。 arXiv:2405.04434，2024。

原题名与书目信息：[4] DeepSeek-AI et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024.

[5] W. Kwon et al. 使用 PagedAttention 进行大型语言模型的高效内存管理。 SOSP 2023。DOI：10.1145/3600006.3613165； arXiv:2309.06180.

原题名与书目信息：[5] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. DOI:10.1145/3600006.3613165; arXiv:2309.06180.

[6] NVIDIA Transformer Engine documentation. PyTorch 注意力机制（Attention）/推理（Inference）参数；解码时K/V缓存。 2026 年 9 月咨询。

原题名与书目信息：[6] NVIDIA Transformer Engine documentation. PyTorch attention/inference parameters; K/V caching during decoding. Consulted September 2026.

[7] vLLM documentation. 自动前缀缓存（Prefix Caching）、缓存配置和前缀缓存隔离/安全。当前文档查阅于 2026 年 9 月。

原题名与书目信息：[7] vLLM documentation. Automatic Prefix Caching, cache configuration, and prefix-cache isolation/security. Current documentation consulted September 2026.

[8] NVIDIA TensorRT-LLM documentation. 键值缓存（KV Cache）系统、重用、优先驱逐和主机卸载（Offloading）。当前文档查阅于 2026 年 9 月。

原题名与书目信息：[8] NVIDIA TensorRT-LLM documentation. KV Cache System, reuse, prioritized eviction, and host offloading. Current documentation consulted September 2026.

[9] vLLM documentation. KV 卸载（Offloading）使用指南和多层卸载 API。当前文档查阅于 2026 年 9 月。

原题名与书目信息：[9] vLLM documentation. KV Offloading Usage Guide and multi-tier offloading APIs. Current documentation consulted September 2026.

[10] vLLM documentation. 量化 键值缓存（KV Cache）和 CacheConfig 存储数据类型（Dtype）选项。当前文档查阅于 2026 年 9 月。

原题名与书目信息：[10] vLLM documentation. Quantized KV Cache and CacheConfig storage dtype options. Current documentation consulted September 2026.

[11] C. Hooper et al. KVQuant：利用 键值缓存（KV Cache）量化（Quantization）实现 1000 万上下文长度 LLM 推理（Inference）。 arXiv:2401.18079，2024。

原题名与书目信息：[11] C. Hooper et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv:2401.18079, 2024.